

Contents

Chapter 16: Specification Engineering	1
1. From Requirements to Specifications	2
2. Why Vague Prompts Fail	3
3. Specification as a Constraint System	4
4. Requirements	5
Functional requirements	5
Non-functional requirements	5
Security requirements	5
Operational requirements	6
5. Invariants	6
6. Acceptance Criteria	7
7. Interfaces	7
8. Constraints	8
9. Tests as Executable Specifications	9
10. Architecture Decision Records	10
11. Specification Engineering vs. Prompt Engineering	11
12. Specification as an Interface Between Humans and Agents	12
13. The Specification-to-Implementation Pipeline	13
14. Measuring the Value of Specification	14
Prompt A — Vague	14
Prompt B — Precise	14
15. The Experiment	15
16. Specification Quality Is an Engineering Variable	16
17. From Requirements Engineering to Specification Engineering	16
18. The Deeper Idea: Specification as Search-Space Reduction	17
19. Exercise — Specification A vs. Specification B	18
20. Key Takeaways	19

Chapter 16: Specification Engineering

If Chapter 15 was about understanding **how coding agents execute software-engineering work**, Chapter 16 addresses a deeper question:

What exactly should the agent build?

This question becomes dramatically more important as coding agents become more capable.

A weak specification forces the agent to infer missing decisions.

A strong specification makes those decisions explicit.

Compare:

“Build me a RAG application.”

with:

“Build a RAG service with these interfaces, constraints, tests, latency targets, security requirements, and acceptance criteria.”

The second statement does not merely provide more detail.

It changes the **engineering problem**.

The agent now has a constrained space of acceptable solutions rather than an open-ended request.

This is the central idea of **specification engineering**:

Transform an intent into a precise, testable, machine-actionable description of the system that should exist.

1. From Requirements to Specifications

Traditional software engineering often begins with requirements:

User needs
↓
Requirements
↓
Architecture
↓
Implementation
↓
Tests

In an agentic development environment, the specification becomes much more important because the implementation may be generated or substantially assisted by AI:

Problem
↓
Specification
↓
Agent
↓
Implementation
↓
Verification

The specification becomes the **contract between human intent and machine execution**.

A useful abstraction is:

$$S = (R, I, C, A, T, D, \dots)$$

where:

- R = requirements
- I = invariants
- C = constraints
- A = acceptance criteria
- T = tests
- D = architectural decisions

The specification defines the set of acceptable implementations:

$$\mathcal{J}_{valid} = \{ i \mid i \models S \}$$

The objective of engineering is no longer simply:

generate code

but:

find implementation i such that $i \models S$

This is a much more precise formulation.

2. Why Vague Prompts Fail

Consider:

Build me a RAG application.

There are hundreds of reasonable interpretations.

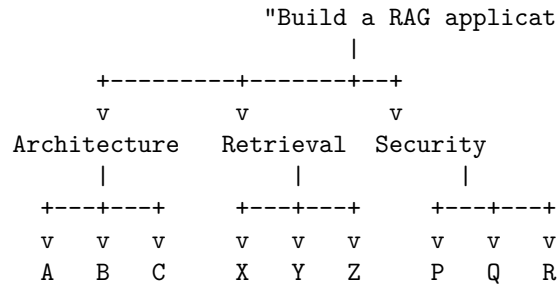
Should it use:

- PostgreSQL or a vector database?
- dense retrieval or hybrid retrieval?
- which embedding model?
- which chunking strategy?
- what document formats?
- what authentication mechanism?
- what API?
- what latency?
- what concurrency?
- what security model?
- what citation requirements?
- what happens when retrieval fails?
- how should hallucinations be handled?
- what evaluation dataset?
- what deployment environment?

The agent must infer all of these.

That creates **specification entropy**.

The larger the number of unspecified decisions, the larger the solution space:



The agent is effectively performing two tasks simultaneously:

1. interpreting the request
2. implementing the system

That is dangerous.

The model may make perfectly reasonable decisions that nevertheless violate the user's actual intent.

A precise specification reduces this ambiguity.

3. Specification as a Constraint System

One useful way to think about a specification is as a collection of constraints.

Suppose the system must satisfy:

$$C_1 = \text{REST API exists}$$

$$C_2 = \text{P95 latency} < 500ms$$

$$C_3 = \text{all API requests require authentication}$$

$$C_4 = \text{responses contain source citations}$$

$$C_5 = \text{retrieval recall} > 0.90$$

C_6 = no document crosses tenant boundaries

The implementation is acceptable only if:

$$C_1 \wedge C_2 \wedge C_3 \wedge C_4 \wedge C_5 \wedge C_6$$

are satisfied.

This is a major conceptual improvement over:

“Build a good RAG application.”

“Good” is subjective.

A specification converts subjective expectations into **observable properties**.

4. Requirements

Requirements describe what the system must accomplish.

A useful distinction is between different classes of requirements.

Functional requirements

What the system does.

For example:

The system shall accept PDF documents.

The system shall index uploaded documents.

The system shall answer natural-language queries.

The system shall return citations.

Non-functional requirements

How the system must behave.

P95 query latency < 500 ms.

Availability > 99.9%.

Maximum document size = 100 MB.

Security requirements

All API endpoints require authentication.

Users may access only documents belonging to their tenant.

Secrets must not be stored in source control.

Operational requirements

Every query must emit a trace ID.
Failures must be logged.
Metrics must be exported.

The distinction matters because agents tend to optimize for **functional completion** unless the specification explicitly includes the other dimensions.

An agent may successfully implement:

"Answer questions over documents."

while completely failing:

"Answer questions securely under multi-tenant isolation."

5. Invariants

An invariant is a property that must remain true throughout system operation.

This is different from a feature.

For example:

Users must never retrieve another tenant's documents.

That is a security invariant.

Formally:

$$\forall u, d : \text{tenant}(u) \neq \text{tenant}(d) \Rightarrow \text{accessible}(u, d) = \text{false}$$

Another example:

Every generated answer must be grounded in retrieved evidence.

Conceptually:

$$\text{Answer}(x) \Rightarrow \text{Evidence}(x) \neq \emptyset$$

Invariants are particularly valuable for agentic systems because they constrain implementation choices.

A feature says:

"Support document retrieval."

An invariant says:

"Document retrieval must never violate tenant isolation."

The second statement eliminates entire classes of otherwise plausible implementations.

6. Acceptance Criteria

Acceptance criteria answer:

How do we know the implementation is finished?

For example:

Feature:

Document upload

Acceptance criteria:

1. PDF files can be uploaded.
2. Files larger than 100 MB are rejected.
3. Unsupported MIME types return HTTP 415.
4. Uploaded documents become searchable.
5. Uploads from tenant A cannot be retrieved by tenant B.
6. Upload failures return structured error responses.

This creates an explicit boundary between:

implementation incomplete

and:

implementation acceptable

Acceptance criteria should ideally be **observable and testable**.

Avoid:

“The interface should be intuitive.”

Prefer:

“A new user can upload a document and execute a query without documentation.”

Even better, define an automated test.

7. Interfaces

Interfaces define how components interact.

For a RAG service:

POST /documents

GET /documents/{id}

POST /query

DELETE /documents/{id}

A specification should describe:

- input schema
- output schema
- error schema
- authentication
- idempotency
- versioning
- timeout behavior

For example:

POST /query

Request:

```
{
  "query": string,
  "top_k": integer
}
```

Response:

```
{
  "answer": string,
  "citations": [
    {
      "document_id": string,
      "page": integer
    }
  ]
}
```

The interface becomes a **contract**. The agent can implement internally however it wants, provided the external contract remains satisfied. This is an important principle:

Specify behavior at the boundary; preserve implementation freedom behind the boundary unless an architectural constraint is intentional.

8. Constraints

Constraints limit the solution space. They may include:

Technology constraints

Python 3.12+
 PostgreSQL
 FastAPI

Resource constraints

`<= 2 GB memory`
`<= 4 CPU cores`

Performance constraints

`P95 latency < 500 ms`

Cost constraints

`LLM inference cost < $0.01/query`

Security constraints

`No plaintext credentials.`
`Tenant isolation required.`

Deployment constraints

`Must run in Kubernetes.`
`Must support horizontal scaling.`

Constraints are especially important for AI agents because otherwise they may optimize for local correctness while violating system-level requirements.

9. Tests as Executable Specifications

One of the most powerful forms of specification is the **executable specification**. Instead of:

The query endpoint should return relevant answers.

write:

```
def test_query_returns_citations():
    response = client.post(
        "/query",
        json={"query": "What is the refund policy?"}
    )
    assert response.status_code == 200
    assert response.json()["answer"]
    assert response.json()["citations"]
```

The test converts a statement of intent into an executable constraint. Conceptually:

$$Specification \rightarrow Test \rightarrow ObservableBehavior$$

This is especially valuable for coding agents. The agent can:

```

read specification
  ↓
implement
  ↓
run tests
  ↓
observe failures
  ↓
repair

```

The specification and verifier therefore form a feedback pair.

10. Architecture Decision Records

Not every decision should become an implementation constraint. Sometimes multiple implementations satisfy the requirements. For example:

Requirement:

Store document embeddings.

Possible implementations:

PostgreSQL + pgvector

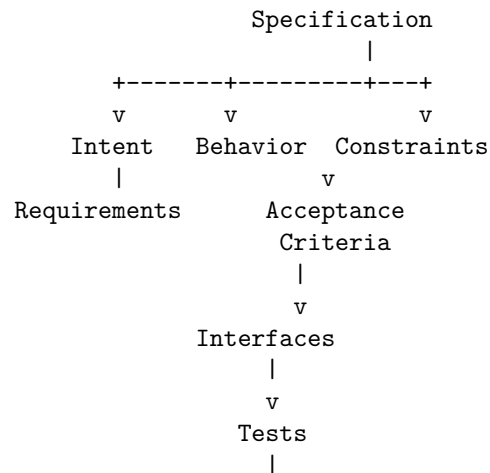
Pinecone

Qdrant

Weaviate

OpenSearch

An **Architecture Decision Record (ADR)** captures why a particular decision was made. A typical ADR contains:



v
ADRs

Each layer answers a different question.

Layer	Question
Intent	Why are we building this?
Requirements	What must it accomplish?
Behavior	How should it behave?
Invariants	What must always remain true?
Interfaces	How do components interact?
Constraints	What limits the solution?
Acceptance criteria	When is it acceptable?
Tests	How can we verify it?
ADRs	Why were architectural choices made?

The layers reinforce one another.

11. Specification Engineering vs. Prompt Engineering

These concepts should not be confused.

- **Prompt engineering** focuses primarily on communicating with a model.
- **Specification engineering** focuses on defining the system to be built.

A prompt might say:

“Implement a secure RAG API.”

A specification might define:

Purpose
Functional requirements
API contracts
Data model
Security invariants
Latency targets
Cost limits
Failure behavior
Acceptance criteria
Test cases
Architecture decisions
Deployment constraints

The prompt is merely one mechanism for transmitting the specification to an agent. This distinction becomes increasingly important as agents become more autonomous. The long-term workflow is unlikely to be:

Human → clever prompt → LLM

It is more likely to be:

Human
↓
Specification
↓
Agent
↓
Planning
↓
Implementation
↓
Verification
↓
Evidence

The specification becomes the durable artifact.

12. Specification as an Interface Between Humans and Agents

There is a deeper architectural implication. Traditional software engineering has interfaces between:

human ↔ human
human ↔ software
software ↔ software

Agentic engineering introduces another:

human ↔ agent

The specification becomes the interface at that boundary. A human may communicate intent in natural language:

“We need a reliable document search service.”

The specification transforms that into something closer to:

Inputs
Outputs
Invariants
Constraints
Failure modes
Acceptance tests
Performance objectives
Security properties

This reduces the amount of implicit reasoning the agent must perform. The human is effectively moving from:

telling the agent what to do

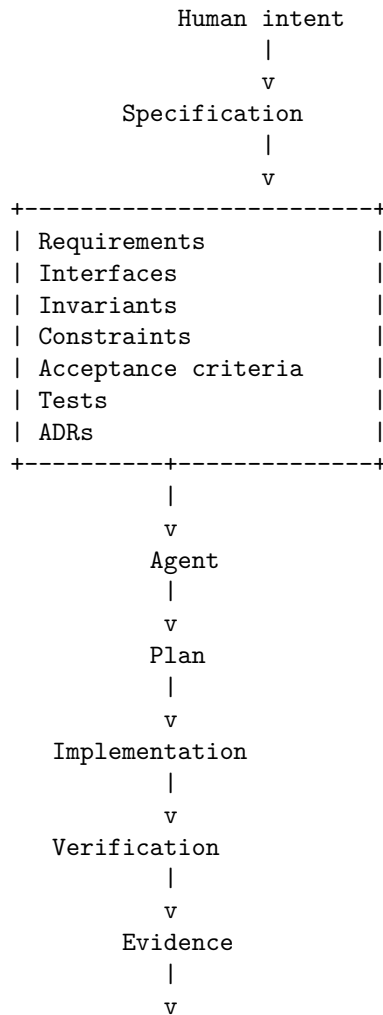
toward:

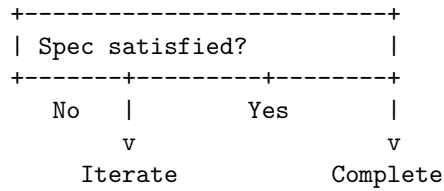
defining the space of acceptable outcomes.

That is a much more scalable interaction model.

13. The Specification-to-Implementation Pipeline

A mature agentic workflow can look like:





The specification is therefore not merely a document produced before coding. It participates in the entire development loop.

14. Measuring the Value of Specification

The most important experiment for this day is empirical. Take one software task. For example:

Build a document-question-answering service.

Give the agent two different specifications.

Prompt A — Vague

Build me a RAG application.

It should ingest documents and answer questions about them.

Prompt B — Precise

Build a Python 3.12 RAG service.

Requirements:

- Accept PDF and Markdown documents.
- Maximum document size: 100 MB.
- Expose POST /documents and POST /query.
- Require authenticated requests.
- Enforce tenant isolation.
- Return citations containing document ID and page/section.
- Use hybrid lexical + semantic retrieval.
- Return top-k configurable results.
- P95 query latency target: <500 ms on the supplied benchmark.
- Return structured errors.
- Do not expose document contents belonging to another tenant.

Acceptance criteria:

- All supplied unit and integration tests pass.
- Unauthorized requests return 401.
- Cross-tenant retrieval tests fail closed.
- Every successful answer contains at least one citation.
- P95 latency remains below 500 ms on the benchmark dataset.

Architecture:

- FastAPI
- PostgreSQL + pgvector
- Redis for caching

The second specification sharply reduces ambiguity.

15. The Experiment

Run the same agent under both conditions. Measure at least:

Correctness

$$Accuracy = \frac{\text{requirements satisfied}}{\text{requirements}}$$

Test success

$$PassRate = \frac{\text{tests passed}}{\text{tests}}$$

Rework

Measure:

number of iterations
 number of failed test runs
 number of reverted edits

Efficiency Measure:

tokens consumed
 tool calls
 wall-clock time
 LLM cost

Specification compliance Create a checklist:

Requirement 1 yes
 Requirement 2 yes
 Requirement 3 no
 Requirement 4 yes
 ...

Architectural compliance Evaluate:

correct framework?
 correct database?
 correct API?
 correct boundaries?

Then compare:

	Vague Spec	Precise Spec
Tests passed		
Requirements met		
Iterations		
Tool calls		
Tokens		
Cost		
Latency		
Security violations		
Rework		
Human corrections		

The experiment turns specification engineering from an abstract idea into a measurable engineering discipline.

16. Specification Quality Is an Engineering Variable

This experiment should lead to a broader conclusion. Suppose agent capability is:

$$Q = f(M, H, S, V, E)$$

where:

- M = model capability
- H = harness quality
- S = specification quality
- V = verification quality
- E = environment/tooling quality

Improving M is not the only way to improve Q . Improving S can be equally important. This creates an interesting possibility:

A weaker model operating against a precise specification and strong verifier may outperform a stronger model operating against an ambiguous specification.

The system architecture determines how much capability can actually be extracted from the model.

17. From Requirements Engineering to Specification Engineering

Traditional requirements engineering remains essential. But agentic development expands its scope. Traditional requirements often emphasize:

What should the system do?

Specification engineering adds:

What must always be true?

What interfaces must exist?

What constraints apply?

How is success measured?

How is failure detected?

What architectural decisions are fixed?

What evidence demonstrates compliance?

This makes the specification much closer to an **engineering contract**. A mature specification may therefore serve simultaneously as:

- a requirements document
- an architectural contract
- an API contract
- a test plan
- an agent instruction set
- a verification target
- a review artifact

This convergence is one of the defining characteristics of AI-assisted software engineering.

18. The Deeper Idea: Specification as Search-Space Reduction

Return to the coding-agent model from Chapter 15. The agent is searching through possible repository states:

$$S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow \dots \rightarrow S_n$$

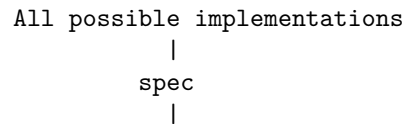
Without a precise specification, the acceptable destination set may be enormous:

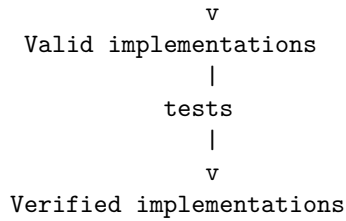
$$\mathcal{G}_{vague}$$

With a precise specification:

$$\mathcal{G}_{precise} \subset \mathcal{G}_{vague}$$

The specification reduces the search space. But this is not necessarily a limitation. It is **useful constraint**.





This is why specification quality directly affects agent performance. The agent does not need to discover every design decision if the specification has already encoded them.

19. Exercise — Specification A vs. Specification B

Choose a problem substantial enough to expose ambiguity. For example:

Build a service that answers questions about a collection of documents.

Create two specifications.

Version A Use only a few sentences. Do not specify:

- technology
- API
- security
- performance
- failure behavior
- tests

Version B Specify:

- functional requirements
- non-functional requirements
- invariants
- interfaces
- constraints
- acceptance criteria
- test cases
- architecture decisions

Run the **same coding agent** against both. Do not change:

- model
- temperature/settings
- repository
- tool access

- verifier
- hardware

Only change the specification. Then compare the resulting systems. Your evaluation should include:

Implementation correctness
 Requirement coverage
 Test pass rate
 Security compliance
 Architecture compliance
 Number of iterations
 Number of tool calls
 Token consumption
 Latency
 Cost
 Human intervention

Finally ask:

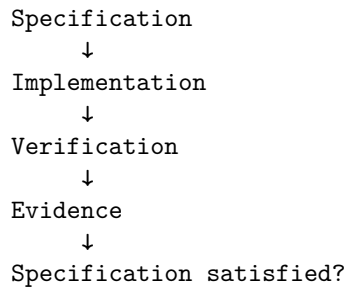
How much of the agent's apparent coding ability was actually specification quality?

That is the central lesson of Chapter 16.

20. Key Takeaways

1. **Specification engineering is the discipline of converting intent into a precise, testable system contract.**
2. **A vague request creates a large solution space.** A precise specification constrains that space and reduces ambiguity.
3. **Requirements describe what the system must accomplish.** They should be complemented by invariants, constraints, interfaces, acceptance criteria, tests, and architectural decisions.
4. **Invariants describe what must always remain true.** They are particularly important for security, correctness, consistency, and multi-tenant systems.
5. **Acceptance criteria define when the implementation is good enough.** Whenever possible, make them observable and executable.
6. **Interfaces turn expectations into contracts.** They allow the agent implementation to evolve while preserving externally visible behavior.
7. **Constraints intentionally reduce implementation freedom.** Technology, performance, security, cost, and deployment constraints prevent locally reasonable but globally unacceptable solutions.

8. **Tests are executable specifications.** They transform requirements into observable evidence and provide feedback for the coding-agent loop.
9. **ADRs provide architectural memory.** They preserve the reasoning behind decisions that an agent might otherwise rediscover or violate.
10. **Specification engineering is different from prompt engineering.** Prompt engineering optimizes communication with a model; specification engineering defines the system and its acceptable outcomes.
11. **The specification is becoming the interface between humans and agents.** Humans increasingly define the desired state and constraints while agents perform the implementation search.
12. **Specification quality directly affects agent performance.** A better specification can improve correctness, reduce iteration, lower cost, and decrease human intervention without changing the underlying model.
13. **Specification and verification form a pair.**



14. **The future software-engineering artifact may be less “code” and more “specification + evidence.”** Code becomes one implementation of a formally constrained desired system.
15. **The key shift is from asking an agent to write software to specifying the state of the world that the agent must create.**

Don’t merely tell the agent what to code. Define what must be true when the work is finished.